

Правила оформления технической документации

Создание и редактирование технической документации

Техническая документация (ТД) формируется по продукту, его подсистемам и сервисам.

Для сервиса имеет смысл оформлять отдельную техническую документацию, если он может быть переиспользован в других подсистемах — как минимум, это позволит удобно организовать перекрестные ссылки между ТД сервиса и подсистем. Если сервис узкоспециализированный и точно будет задействован только в одной-единственной подсистеме, то достаточно добавить информацию о нем в ТД подсистемы.

Процесс создания технической документации — это процесс итеративный. Выстраивайте каркас документации, в который не забывайте заселять новые сведения и актуализировать старые.

Пишите техническую документацию так, чтобы любой новый человек в вашей команде с легкостью разобрался, как работает ваш продукт и его подсистемы.

Техническое задание (ТЗ) ни в коем случае не может выступать технической документацией. ТЗ может быть только источником сведений для технической документации. В техническую документацию переносится только часть информации из ТЗ (например, критерии сдачи переносить в техническую документацию не следует). Ссылки из технической документации на технические задания по проектам недопустимы, необходимая информация из ТЗ должна быть отражена непосредственно в ТД.



Если по продукту или по его подсистеме в рамках какого-либо проекта была создана техническая документация, в рамках новых проектов следует лишь дополнять имеющуюся. Создавать техническую документацию заново запрещается.

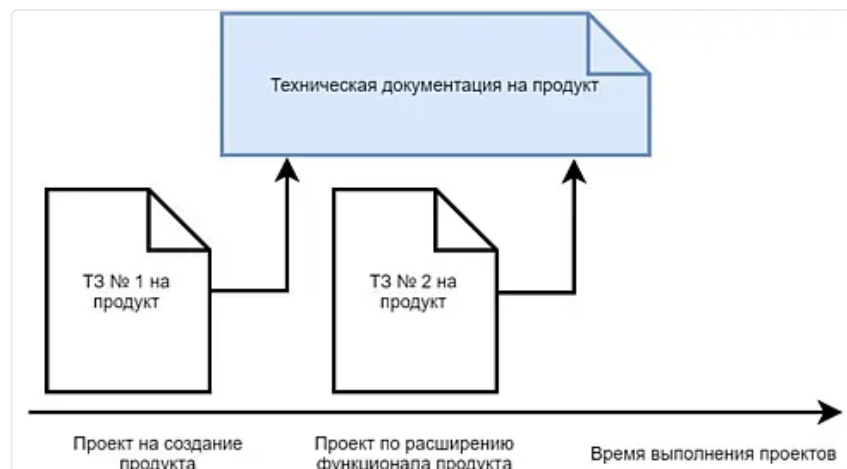


Рисунок 1. Принцип формирования технической документации

Формат и место хранения

Формат — документ Saby (*.sabydoc)

Место хранения документации — **база знаний разработки продукта.**

Ссылки на техническую и пользовательскую документацию должны быть указаны в описании участка Кайдзен согласно [правилам организации участков работ](#).

Структура документации

Структура документации зависит от описываемого продукта.

Для **прикладных продуктов** важно внутреннее устройство, поэтому в явном виде не выделяется папка Техническая документация: все, что рассказывается о прикладном продукте, — это его техническая документация.

Для **внутренних продуктов**, и особенно для платформенных переиспользуемых решений, важнее рассказать, как использовать этот инструмент в прикладные области, поэтому статьи с описанием, как встраивать и инструкции/

регламенты вынесены в корень документации, а описание внутреннего устройства вынесено в отдельную папку Техническая документация.



Рекомендованный [шаблон базы знаний разработки прикладного продукта](#) и [шаблон базы знаний разработки внутреннего продукта](#)

Ниже представлена таблица с пояснениями к каждому разделу (статье) документации.

Раздел документации	Условие присутствия	Цель	Шаблон	Кто читает
<Продукт>				
Описание	Обязательный	Дать общее представление об участке: решаемые задачи, как устроен "изнутри", какие сущности данных есть на участке.	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/c85fab44-5cb8-4b6f-acdd-ab8668f8e77c	Все
База данных	Если продукт состоит из подсистем, у каждой из которых своя база данных, то раздел можно не заполнять, но описание БД должны быть представлено в ТД подсистем. Если у всех подсистем продукта есть общая часть, то ее правильнее описать в БД продукта, а в документации подсистемы — детальное описание БД с сущностями, присущими только этой подсистеме.	Описать структуру базы данных продукта	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/33cacc13-f243-4976-bfa6-ee88834ada72	- Внутренняя команда - Внешние отделы Разработки - Управление
<Подсистема>				
Описание	Обязательный	Дать общее представление об участке: решаемые задачи, как устроен "изнутри", какие сущности данных есть на участке.	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/54f84359-65b7-4210-95a9-bcd661b0b7dc	Все
Как встраивать (описание внешнего API)	Требуется, если подсистема предоставляет API для внешних отделов Разработки	Предоставить инструкции по встраиванию участка в смежную подсистему	Если достаточно просто перечислить методы API — используйте https://online.sbis.ru/shared/disk/878c62ac-d895-4b2b-	- Внутренняя команда - Внешние отделы Разработки - Управление

			832d-d6e29338a29d - Если порядок встраивания более сложный чем вызов метода API, оформите пошаговую инструкцию в произвольном виде Пример: https://link.sbis.ru/wasabybackend/knowledge?published=true&mode=readList&article=aa038d24-6891-487b-b9c6-cf7cc2e96c5c - Для интерфейсных компонентов - используйте https://online.sbis.ru/shared/disk/3b83854c-f59d-4f10-ab35-509ad677f662 Пример: https://link.sbis.ru/article/wasaby/14b8c2f4-296b-44ea-8110-2bcf4fc8c7b6	
Архитектура	Требуется для внутренних продуктов	Более детально описать внутреннюю архитектуру подсистемы	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/73efb2fd-5caf-43ae-b9b7-e9e9234beb16	- Внутренняя команда - Управление
База данных	Обязательный	Описать структуру базы данных	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/53ff5d3d-48e5-41af-aca5-1f6575c46d1a	- Внутренняя команда - Внешние отделы Разработки - Управление
Алгоритмы и процессы	Необязательный	Раскрыть подробнее отдельные процессы взаимодействия участников архитектуры	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/908fd751-96c8-46ac-ab74-0bb81047073a	Внутренняя команда
Архитектура интерфейса	Требуется, если на участке есть интерфейс	Описать декомпозицию интерфейса на контролы, а также	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/782b33	- Внутренняя команда - Управление

		цепочки вызовов методов БЛ при построении страницы	94-9def-476a-8723-bb85e93208ea	
Особенности работы интерфейса	Требуется, если в интерфейсе есть сложная логика показа или расчета данных, элементов страницы	Документ-шпаргалка для менеджера проекта, проектировщика, технолога, в котором есть ответ на вопросы: "что за показатель тут выводится?", "как посчитали эту цифру?" и т.п.	В произвольном виде Примеры: https://link.saby.ru/ZKcMPF9BQqCLD4ieL_ad-0g/knowledge?article=87543298-6197-4c91-a308-54f343678e3f&published=true&mode=readList https://link.sbis.ru/article/dev-accounting/6ce8373a-e23c-47f9-9c0a-4e2a781a02f9	- Внутренняя команда - Технологи
Инструкции по эксплуатации	Требуется, если у подсистемы есть админки или нужно опубликовать инструкции/ регламенты по использованию подсистемы для сотрудников Тензора	Предоставить инструкции по эксплуатации подсистемы	В произвольном виде Пример: https://link.sbis.ru/saby/knowledge?article=d0c687b5-76fb-4e52-a312-dd42f8b39002&published=true&mode=readList	- Внутренняя команда, - Внешние отделы Разработки, - ЦОД - Сборка
Организация кода	Обязательный	Справочная информация о месте размещения кода, использовании сторонних библиотек.	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/8dff614c-727b-4a8d-93ef-f286d9665df5	- Внутренняя команда - Внешняя команда
Подсистема распределения прав	Требуется, если функционал участка (клиентских сценариев, админок) ограничиваются правами	Описать, какие в участки системы, ограничения предусмотрены для участка; в какие роли какие права на участок включены.	В произвольном виде	- Внутренняя команда - Технологи - Управление - ЦОД
Параметры облака, задачи планировщика	Требуется, если на участке используется конфигурирование параметрами облака и используется планировщик задач	Зафиксировать, какие параметры облака и задачи планировщика используются и для чего	В произвольном виде	- Внутренняя команда - ЦОД - Сборка
<Облачный сервис>	Требуется, если в состав подсистемы входит прочий облачный сервис, реализованный для нее	Описать облачный сервис, который является инфраструктурой описываемой подсистемы	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/3b25d6ef-85b4-4ec0-abc5-4bbe5f18dfa6	- Внутренняя команда - Внешняя команда - Управление - ЦОД

<Продукт> в мобильном приложении				
Описание	Обязательный	Дать общее представление об участке: решаемые задачи, как устроен "изнутри"	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/39e4fea6-0273-441b-ac72-cd9c7602345b	Все
Микросервис <название микросервиса>	Обязательный	Описать микросервис контроллера мобильного приложения	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/41296379-839c-4054-83aa-5bbc9733b11a	- Внутренняя команда - Внешние отделы Разработки - Управление
Архитектура интерфейса	Обязательный	Описать декомпозицию интерфейса на контролы	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/32e4a7c7-c407-4286-8cc1-3a7bf4098f3c	- Внутренняя команда - Управление
Особенности работы интерфейса	Требуется, если в интерфейсе есть сложная логика показа или расчета данных, элементов страницы	Документ-шпаргалка для менеджера проекта, проектировщика, технолога, в котором есть ответ на вопросы: "что за показатель тут выводится?", "как посчитали эту цифру?" и т.п.	В произвольном виде	- Внутренняя команда - Технологи
Мобильное приложение <название приложения>	Требуется, если у продукта есть приложение, в котором продукт является флагманским	Описывает задачи приложения и состав его контроллера	https://online.sbis.ru/article/a3742087-2b72-4a61-8baa-860011943a75/e9d2b8f7-2c40-46b8-9c46-98d68dc5d06e	- Внутренняя команда - Внешние отделы Разработки - Управление

Рекомендации по организации структуры

1. Разбивайте техническую документацию по отдельным файлам sabydoc: как минимум, разделы первого уровня рекомендуем организовывать отдельными файлами.
2. Если продукт/подсистема состоит из крупных изолированных кусков функционала (они фактически тоже являются подсистемами, только более мелкими), под каждую такую подсистему рекомендуем выделить папку внутри.
3. Шаблон — это лишь способ подсказать, что нужно рассказать в документации и как ее выстроить так, чтобы она была понятна. Но помните, что никакой шаблон не должен быть препятствием сделать вашу документацию логичной, понятной, информативной. К тому же шаблон на "все случаи жизни" мы предложить не сможем. Поэтому если вы считаете, что в вашей документации абсолютно точно должна быть какая-то информация, которая не предусмотрена шаблоном, смело добавляйте ее. Но не надо вписывать ее в какое-то, как вам кажется, более-менее подходящее место в рекомендованных шаблонах статей — как правило, это приводит к тому, что они превращаются в огромные "простыни" из разрозненных тем. Лучше сделайте отдельную статью.
4. Обратите внимание, что в таблице выше для продукта рекомендованных разделов всего два: описание и база данных, и то не всегда. Часто этого достаточно для продукта — все остальное будет в ТД его подсистем. Но если

вы понимаете, что для вашего продукта важно рассказать архитектуру продукта в целом, есть API или можно указать ссылку на репозиторий с кодом, то можно добавлять в ТД продукта необходимые разделы, предусмотренные в шаблоне ТД подсистемы.

5. В Материалах должно храниться только то, что, по сути, нельзя корректно сконвертировать в `sabydoc`: таблицы Excel, презентации, оригиналы схем и т.п. Все остальное, что является частью технической документации, должно быть размещено в Базе знаний. Это, как минимум, даст возможность найти эти статьи поиском по базе(-ам) знаний.
6. Сравнение с конкурентами в виде отдельного документа — также в Материалах группы.
7. Документ с описанием Бизнес-процесса должен храниться в [специальной базе знаний](#). Для быстрого доступа из группы ссылку на него можно добавить в Материалы группы.

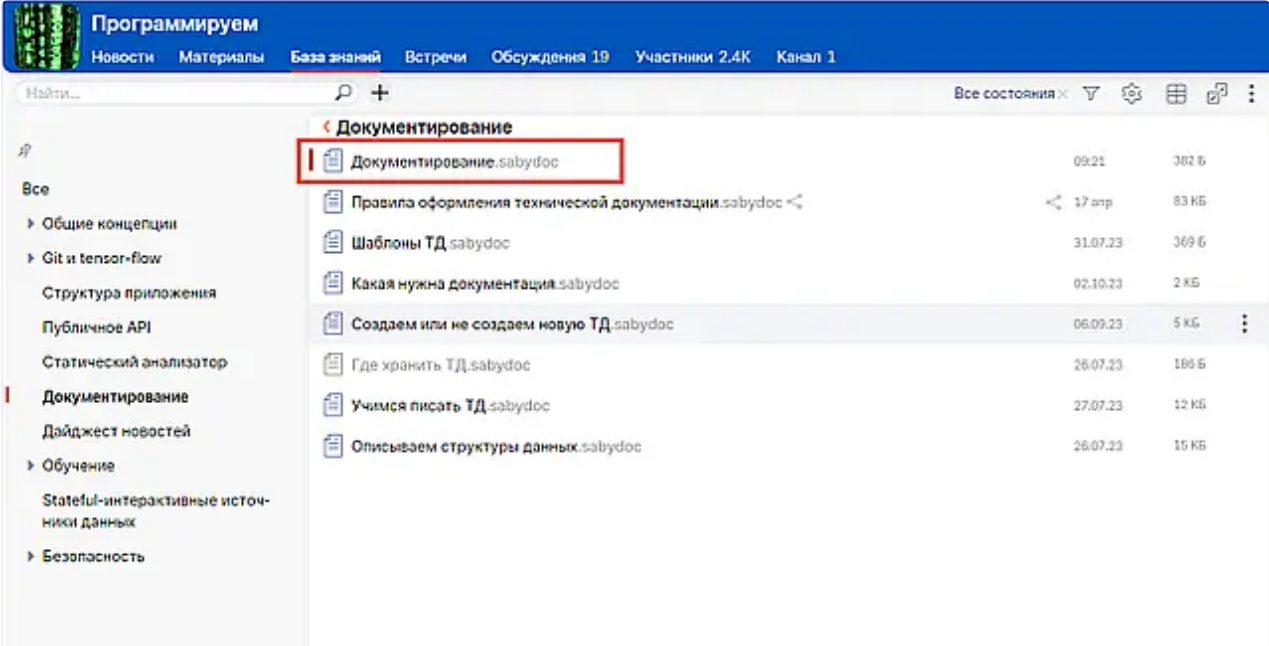
Рекомендации по оформлению статей в Базе знаний

1. Перекрестные ссылки на статьи размещайте на подходящей фразе, чтобы текст не выглядел перегруженным из-за декорированных ссылок.

Исключением могут быть только ситуации, когда важно показать, куда именно ведет ссылка.

2. Используйте заголовочные статьи — текст, который будет размещен на разделе в режиме "Ленты".

Без заголовочной статьи



The screenshot displays the 'База знаний' (Knowledge Base) interface within the 'Программируем' (We Program) section. The top navigation bar includes links for 'Новости', 'Материалы', 'База знаний', 'Встречи', 'Обсуждения 19', 'Участники 2.4K', and 'Канал 1'. The 'База знаний' tab is active, showing a search bar and a list of documents under the 'Документирование' (Documentation) category. The first document, 'Документирование.sabydoc', is highlighted with a red box. The list includes various documents related to technical documentation, such as 'Правила оформления технической документации.sabydoc', 'Шаблоны ТД.sabydoc', and 'Какая нужна документация.sabydoc'.

Документ	Дата	Размер
Документирование.sabydoc	09.21	382 Б
Правила оформления технической документации.sabydoc	17 апр	83 КБ
Шаблоны ТД.sabydoc	31.07.23	369 Б
Какая нужна документация.sabydoc	02.10.23	2 КБ
Создаем или не создаем новую ТД.sabydoc	06.09.23	5 КБ
Где хранить ТД.sabydoc	26.07.23	186 Б
Учимся писать ТД.sabydoc	27.07.23	12 КБ
Описываем структуры данных.sabydoc	26.07.23	15 КБ

Программируем

Новости Материалы База знаний Встречи Обсуждения 19 Участники 2.4K Канал 1

Найти...

Документирование

Документирование

В этом разделе собраны рекомендации по оформлению и размещению документации разработки.

Документирование

Правила оформления технической документации

Создание и редактирование технической документации

Техническая документация (ТД) формируется по продукту, его подсистемам и сервисам.

Для сервиса имеет смысл оформлять отдельную техническую документацию, если он может быть переиспользован в других подсистемах — как минимум, это позволит удобно организовать перекрестные ссылки между ТД сервиса и подсистем. Если сервис узкоспециализированный и точно будет задействован только в одной-единственной подсистеме, то достаточно добавить информацию о нем в ТД подсистемы.

Процесс создания технической документации – это процесс итеративный. Выстраивайте каркас документации, в который не забывайте заселять новые сведения и актуализировать старые.

Пишите техническую документацию так, чтобы любой новый человек в вашей команде с легкостью разобрался, как работает ваш продукт и его подсистемы.

Техническое задание (ТЗ) ни в коем случае не может выступать технической документацией. ТЗ может быть только источником сведений для технической документации. В техническую документацию переносятся только часть информации из ТЗ (например, критерии сдачи переносятся в техническую документацию не следует). Ссылки из технической документации на технические задания по проектам недопустимы, необходимая информация из ТЗ должна быть отражена непосредственно в ТД.

Лента без заголовочной статьи

Программируем

Новости Материалы База знаний Встречи Обсуждения 19 Участники 2.4K Канал 1

Найти...

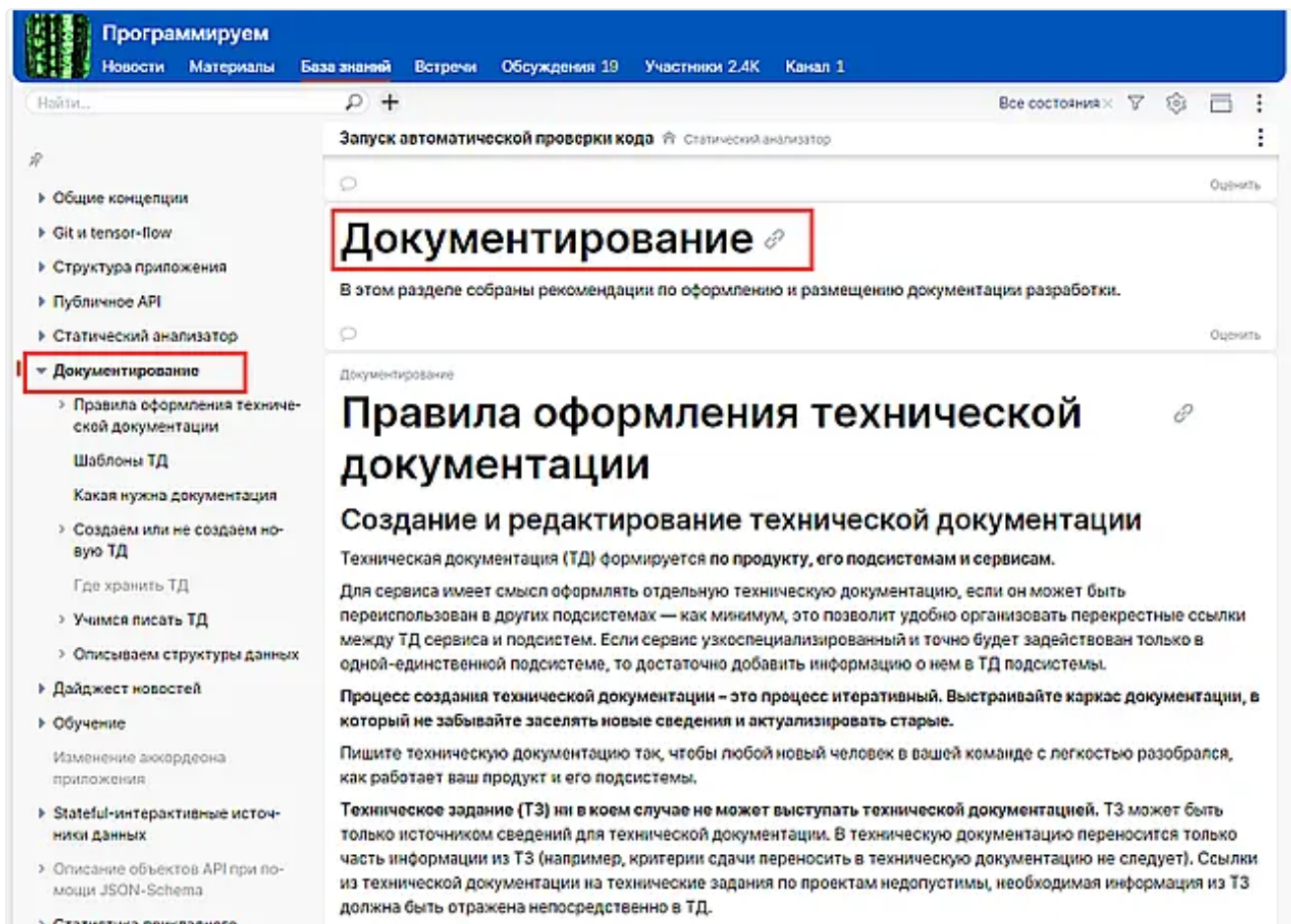
Документирование

- Документирование.sabydoc
- Правила оформления технической документации.sabydoc
- Шаблоны ТД.sabydoc
- Какая нужна документация.sabydoc
- Создаем или не создаем новую ТД.sabydoc
- Где хранить ТД.sabydoc
- Учимся писать ТД.sabydoc
- Описываем структуры данных.sabydoc

- Открыть в новой вкладке
- Поделиться
- В избранные
- Скачать
- Редактировать
- Копировать
- Снять с публикации
- Удалить
- Распечатать
- Переименовать
- В другой раздел
- Назначить заголовочной статьей**
- Теги
- Редакции

Сделать статью заголовочной

С заголовочной статьей



Лента с заголовочной статьей

Стандарт оформления описания

Любая техническая документация должна начинаться с описания. Задача этого документа — дать представление, как устроен участок.

Оформляется по шаблону:

- для подсистемы
- для продукта

Правила оформления схемы взаимодействия сервисов

1. Связь от инициатора. Стрелки всегда должны идти от инициатора процесса, то есть «кто -> кого зовет». Поэтому все «двусторонние» стрелочки сразу вызывают вопросы, и их лучше не использовать. При этом вариант «туда — мы позвали другой сервис по HTTP, обратно - он нам отдал ответ» мы рассматриваем как однонаправленное взаимодействие, поскольку «сам» сторонний сервис нас не звал, и без нас ничего бы самостоятельно не делал.

Фактически, единственным вариантом двунаправленного взаимодействия является какой-то асинхронный обмен данными.

2. Названия сервисов должны быть такими же, как в Структуре облака. Не заставляйте догадываться, о каком сервисе идет речь по названиям-синонимам.
3. Если для концептуального понимания архитектуры важно показать связи сущностей — добавьте их на схему архитектуры. Если схема становится слишком сложной, можно сделать отдельную.
4. Конечные точки. Не нужно рисовать несколько стрелок, приходящих в одну точку, т.к. связь в таком случае непонятна.
5. Группировка. Для лучшего понимания схемы на ней можно выделить группировкой некоторые сущности:
 - набор близких по функционалу сервисов;
 - расположение этих сервисов (логическое — относительно связи с другими сервисами, например, основным, или физическое — у клиента/в нашем ЦОДе);
 - группу однотипных БД.

Пример [схемы архитектуры Чаты в SabyGet](#).

Стандарт описания структуры базы данных

Для описания баз данных используйте [шаблон](#).

Схема сущностей

Схема сущностей представляет краткое описание базы данных и отображает взаимодействие данных между собой.

Схема содержит:

- сущности;
- их логические объединения;
- связи между ними.

Сущности

Сущность — это логически единый элемент структуры базы данных. Обозначается овалом. Название сущности указывается в центре внутри овала по-русски. Если имя таблицы не совпадает с именем сущности или у таблицы английское название, второй строкой указывается оригинальное название таблицы.



Рисунок 3. Элемент структуры – сущность

Логические объединения

Логическое объединение:

- пачка однородных сервисов;
- группа одинаковым образом секционированных таблиц.

Несколько сущностей могут быть объединены по смыслу и представлять логическое объединение. Например: *Документ и событие представляют логическое объединение «Событие по документу».*

Логическое объединение отображается прямоугольником, где в левом верхнем углу указывается его название по-русски.

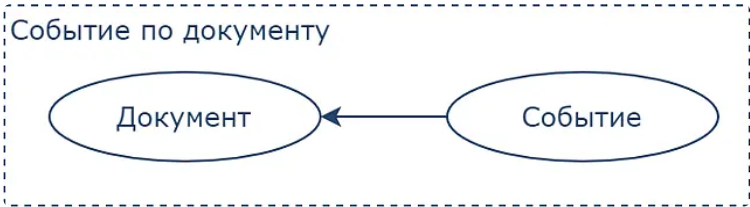


Рисунок 4. Логическое объединение

Связи

Связь обозначает отношение между сущностями. Связи отображаются прямыми стрелками от одной сущности к другой.

На схеме отображаются именно логические связи, а не их физические реализации типа FK. Например, если ваша сущность хранит ID сущности из другого внешнего сервиса, то нарисуйте и тот сервис «рамочкой», сущность в нем и стрелку к ней.

Связи могут быть трёх типов:

	Один к одному
	Многие к одному
*описано ниже (пункт 3)	Многие ко многим

Типы связей

1. Один к одному. Пример: *распределяются задачи между сотрудниками.*

- Каждый сотрудник работает над ровно одной задачей.
- Над каждой задачей трудится ровно один сотрудник.
- Есть задачи, над которыми никто не работает.

- Нет сотрудников, которые ничего не делают.

2. Многие к одному. Пример: отделы и сотрудники.

- В каждом отделе много сотрудников.
- В отделе может не быть сотрудников.
- Отдельный сотрудник может числиться в одном и только в одном отделе.

Связь «Иерархия» — частный случай связи «Многие к одному», где записи находятся в одной таблице. Связь «Иерархия» говорит о том, что объекты из одной таблицы могут ссылаться друг на друга.

Пример: Функциональные области подразделяются на группы. Например, есть группа Дополнительные сервисы, которая в свою очередь делится на подгруппы.

3. Многие ко многим.

Связь «Многие ко многим» реализуется через отдельную таблицу и связи «Многие к одному».

Пример: должности сотрудника.

- у одного сотрудника может быть несколько должностей
- одна должность может быть у нескольких сотрудников

Связь сущностей будет выглядеть так: Сотрудник <— ДолжностьСотрудника —> Должность.



Чтобы разобраться, как правильно и понятно изображать связи на схеме сущностей — обратите внимание на рекомендации в разделе [Как сделать схему понятнее](#)

Правила оформления схемы сущностей и ее описания

1. Схема должна быть простой и понятной без подтекста. И тем не менее, под схемой должно быть краткое описание каждой сущности (буквально 2-3 строки).
2. Минимизируйте "угловые" (90*-angle) связи:
 - Они чисто визуально занимают большее пространство на схеме, чем «прямые», что мешает пониманию.
 - Они имеют свойство перекрываться в самых неожиданных местах, особенно на концах, что иногда делает понимание схемы просто невозможным (см. варианты на картинке).
3. Избегайте пересечений. Располагайте сущности на плоскости схемы, минимизируя количество пересечений связей. Оптимально – вообще обойтись без них. Если без пересечений никак, используйте дугообразные стрелки.
4. Если получается сложная схема с пересечениями – разделите ее на несколько составляющих. Но при этом должна быть общая укрупненная схема, объединяющая все составляющие.
5. Схема должна быть выполнена в сине-белом цвете, допускается добавление фона для элементов структуры, если это помогает пониманию схемы. Для рисования используются элементы из [библиотеки](#). Загрузите ее по [инструкции по работе в Draw.io](#) (раздел «Загрузите библиотеку»).
6. При описании сущностей:
 - не допускайте «рекурсии» с использованием в описании самой сущности (ех: «событие - это событие ...»)
 - не допускайте «кросс-ссылок», то есть ссылок на еще не описанные выше другие сущности.
 - если вы используете уже существующие сущности и храните там ровно то же самое, что и все остальные, то их описывать не надо. Это правило распространяется и на таблицы. Пример — документ.

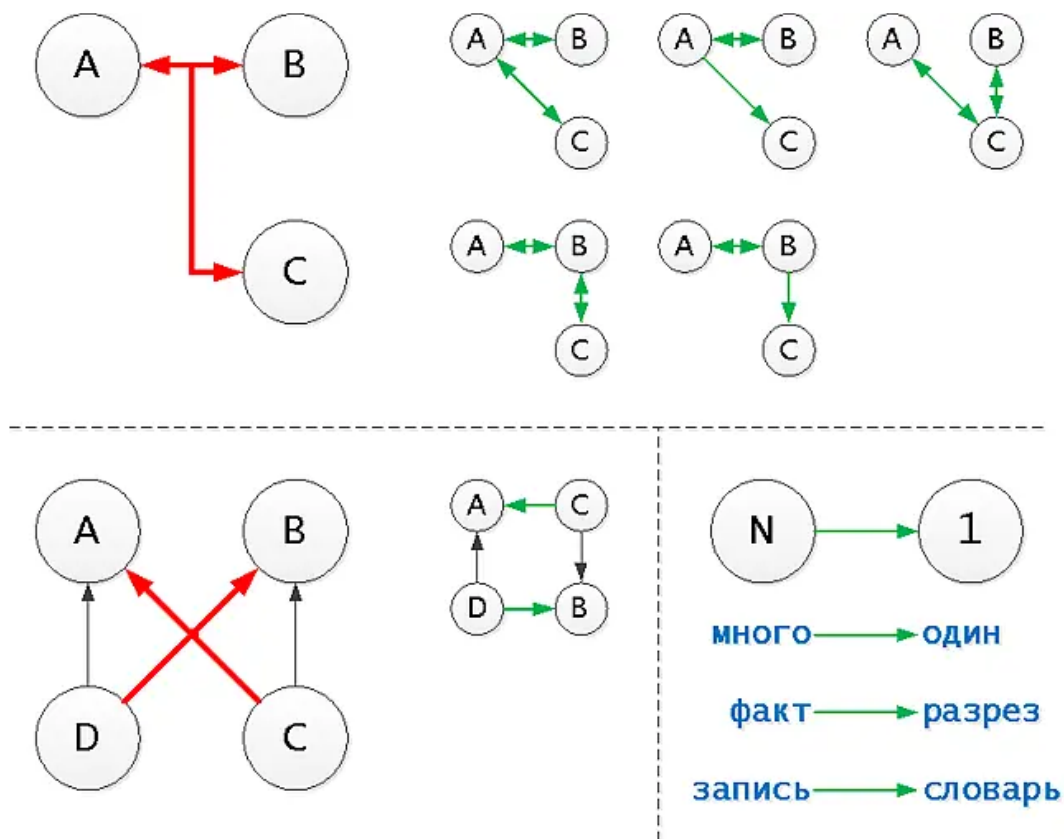


Рисунок 5. Как рисовать связи

Как сделать схему понятнее

Основания и следствия

В нашей системе есть устоявшиеся понятия связей объектов между собой: документов, наименований, лиц...

Традиционно подобные связи "многие-ко-многим" реализуются через отдельную таблицу, причем объект-основание и объект-следствие неравноценны, то есть связь является направленной. Но на рис. 5 непонятно, какой из документов является основанием, а какой – следствием.

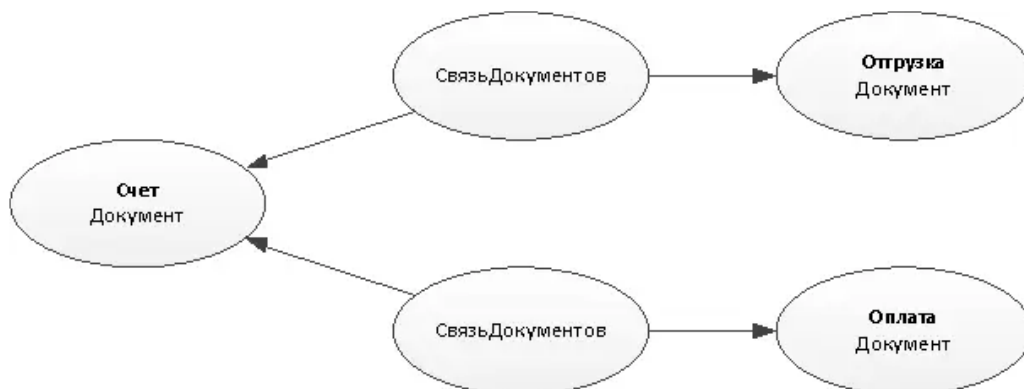


Рисунок 5. Связь Основание-Следствие через таблицу СвязьДокументов

Если же прямо на связи подписать, через какую таблица она осуществляется, то таких вопросов не возникает.

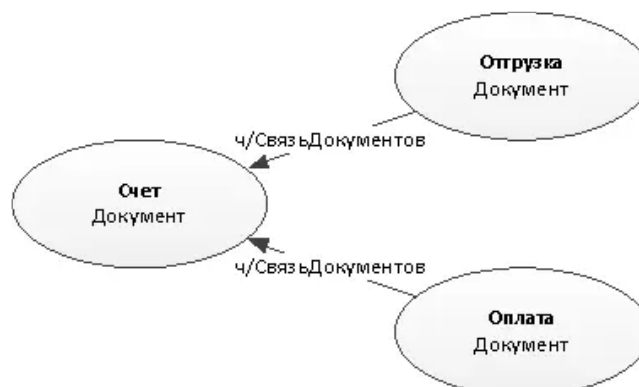


Рисунок 6. Связь Основание-Следствие подписью на связи

Таким образом, рекомендуем:

- на связях писать важную информацию;
- на схеме должен присутствовать только необходимый минимум элементов;
- этих элементов должно быть достаточно для однозначного прочтения.

JSON, содержащий связи

Поскольку мы все-таки стараемся не вводить новых таблиц без надобности, то в некоторых случаях (когда их заведомо немного и нужны все и сразу) связи объектов можно положить в JSON-массив в одном из полей.

На схеме отразите реальную логику связей — где 1-1, где N-1 — и укажите поле, где предполагаете хранить связь.

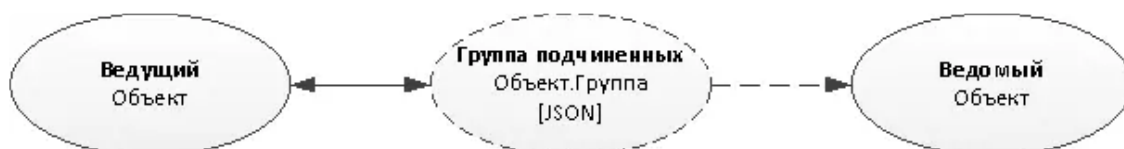


Рисунок 7. Связь в JSON-массиве

Разные логические сущности в одной таблице

Разделяйте логические сущности и правильно их называйте, даже если физически они находятся в одной таблице.

Разные типы данных

К примеру, если у сервиса часть данных персональные (имеют логическую связь с аккаунтом), а часть — глобальные для сервиса (отличаются отсутствием связи с аккаунтом), разделите их на несколько элементов схемы, указав, что это все записи таблицы «Данные».

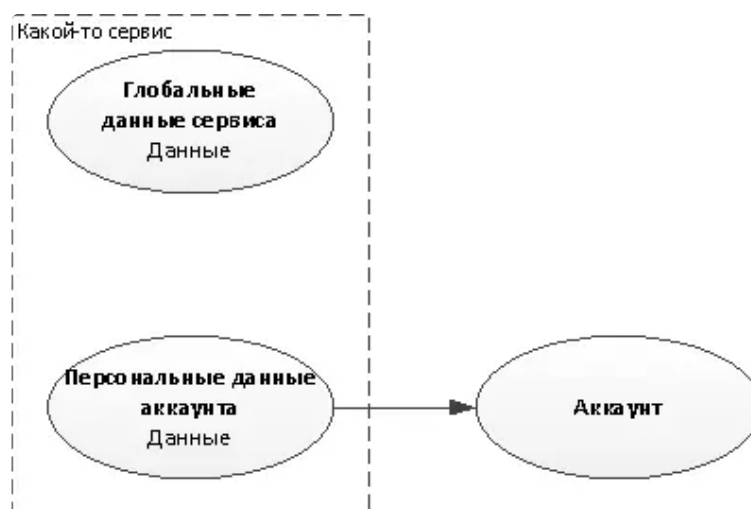


Рисунок 8. Объекты с разным типом данных в одной таблице

Разные по смыслу объекты

Объекты «Ведущий» и «Ведомый» хоть и лежат в общей таблице, но имеют разную логику связей, которую лучше нарисовать явно.

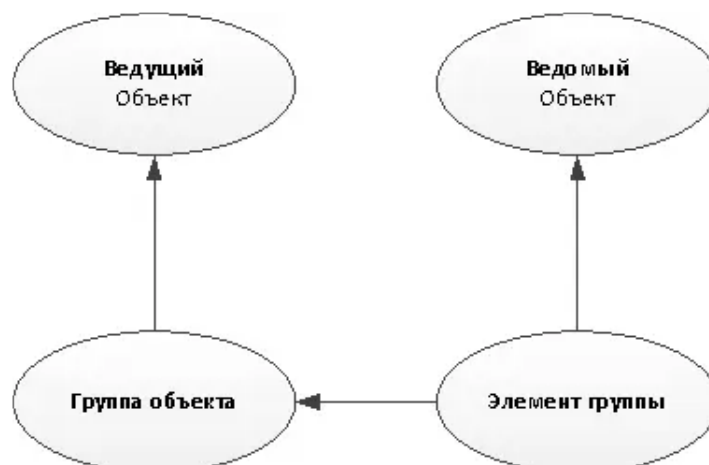


Рисунок 9. Разные по смыслу объекты в одной таблице

Разные по связям объекты

В некоторых случаях объекты, находящиеся в одной таблице, вообще не имеют никаких отношений между собой и принципиально отличаются логикой связей (на рис.10 – объекты, которые физически записи в таблице «Документ»).

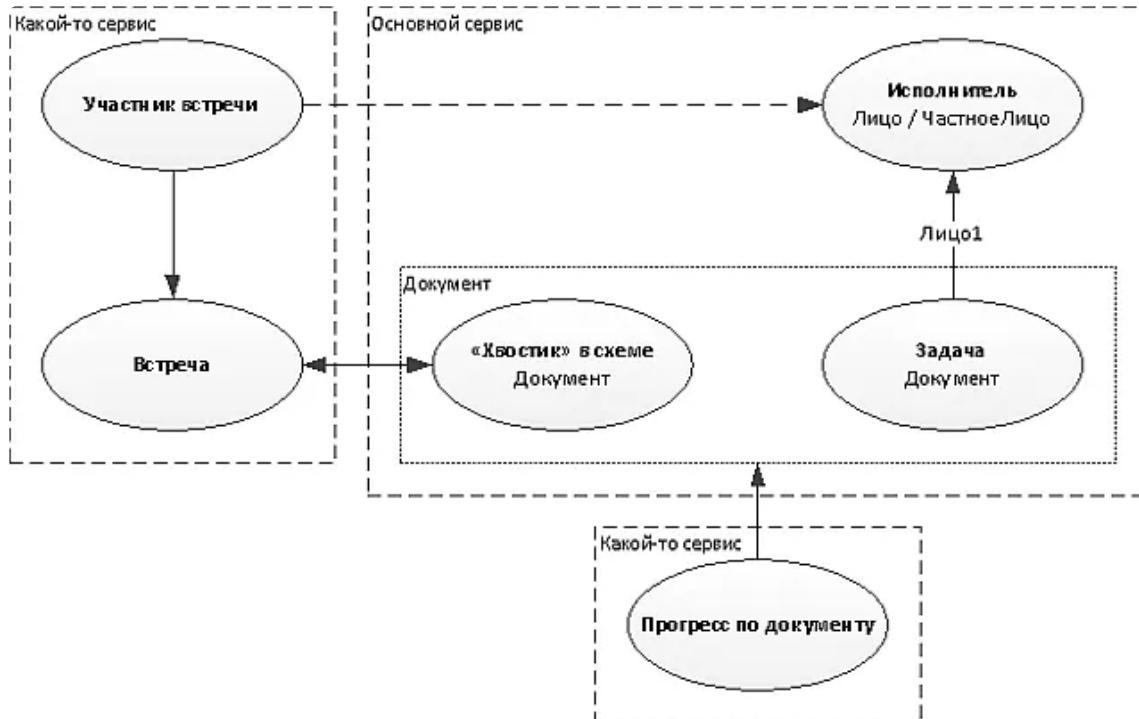


Рисунок 10. Разные по связям объекты

Примеры схемы сущностей

[Схема сущностей Табели](#)

[Схема сущностей Конфигуратор в телефонии](#)

[Схема сущностей Социальная сеть](#)

[Схема сущностей Сервис агрегации сделок](#)

Типовые выборки

При описании типовой выборки надо указать:

- <Номер выборки>;
- <Название выборки>;
- <Функциональное описание>;
- <Алгоритм построения>.

Примеры выборки

Получение информации о группе при отображении в интерфейсе страницы

Для отображения в интерфейсе необходима информация, находящаяся в таблице «Группа». При запросе будет передан конкретный идентификатор группы.

Алгоритм выборки:

1. Выбираем шард по входному параметру метода «Группа». Ключ шардирования – «Группа».
2. Делаем запрос к таблице «Группа». Доступ по индексу («Группа»).

Получение списка моих групп для отображения в интерфейсе

Список групп в интерфейсе будет выводиться постранично, с сортировкой по убыванию даты вступления в группу.

Алгоритм выборки:

1. Выбираем шард по идентификатору персоны, вызвавшей метод. Ключ шардирования – «Получатель».
2. Из таблицы «Подписчик» получаем список идентификаторов групп, членом которой является персона с сортировкой по убыванию даты вступления в группу. Доступ по «Получатель, Канал».
3. Разбиваем список групп по шардам.

- Параллельно для каждого шарда запрашиваем информацию из таблицы «Группа» («доступ по «Группа»»), количество участников группы из таблицы «Подписка» (доступ по «Канал, Получатель»).

Получение списка групп, в которых персона еще не состоит, но доступных для отображения в поиске

Список групп в интерфейсе будет выводиться постранично с сортировкой по убыванию даты создания группы. Применение классического механизма LIMIT OFFSET невозможно, так как таблица «Группа» шардируется по уникальному идентификатору группы. При запросе данных из интерфейса должна быть передана информация о дате и времени создания последней отображенной группы, а также о числе записей.

Алгоритм выборки:

- Выбираем шард по идентификатору персоны, вызвавшей метод. Ключ шардирования – «Получатель».
- Из таблицы «Подписчик» получаем список идентификаторов групп, членом которой является персона – МоиГруппы. Доступ по (Получатель, Канал).
- В методе передается информация о дате и времени создания группы, которая была отражена в интерфейсе последней, а также информация о количестве записей для выборки. Во всех шардах запрашиваем информацию о группах, где Группа NOT IN (МоиГруппы), ТипГруппы IN (1,2), ДатаСоздания<ДатаСозданияПоследняя (последняя дата создания, отображенная в интерфейсе), LIMIT {LIMIT}. Доступ по индексу (ДатаСоздания DESC);
- Объединяем информацию из шардов и выбираем TOP {LIMIT}.

Описание индексов

Формат описания индекса: **<Имя таблицы>(<поля индекса>) [WHERE <условие применимости>]**

Пример:

Документ(Раздел, Раздел@) WHEREРаздел IS NOT NULL

Документ(ИдентификаторДокумента)

Почему именно такой формат:

- Имя индекса никого, кроме, может быть, вас самих, не интересует. А вот имя таблицы, на которую вы его накладываете, очень важно.
- Условие применимости индекса — это совсем не то же самое, что bool-поле в списке его полей.
idx(a, b IS NULL)заполняется для всех записей таблицы, в отличие от idx(a) WHERE b IS NULL. И вот тут уже даже догадаться нереально, что конкретно подразумевал автор.

Особые типы индексов

Тип индекса в наших продуктах — btree в 99.5% случаев, поэтому именно его указывать не надо. Все остальные являются исключениями, и их — надо! Указывайте не-btree тип индекса перед его описанием в явном виде: gin : Лицо(to_tsvector('simple'::regconfig, "Название"))).

Правила описания интерфейсных проектов

Если интерфейс продукта /системы разрабатывается с нуля или в него вносятся значительные изменения, обязателен раздел технической документации «Архитектура интерфейса».

В нем необходимо показать декомпозицию интерфейса, указать какие компоненты были дописаны плюсом к платформе и описать алгоритм построения страниц – обозначить:

- что строится на сервере, что на клиенте и почему;
- есть ли «прелоадинг» на страницах, и на каких именно;
- какие данные откуда грузятся: какие «прилетают» сразу, а какие грузятся динамически и почему;
- что берется из record-set, а что запрашивается из облака;
- какие данные грузятся при нажатии тех или иных кнопок/выборе фильтров в интерфейсе;
- и другие особенности построения и разработки страниц, которые вы сами хотели бы выделить.

Рекомендуется оформлять описание по [шаблону](#).

Правила описания API

Описание API оформляется по [шаблону](#).

Рекомендуются использовать вместо статичного описания методов ссылки на раздел [автодокументации бизнес-логики](#) или карточки методов в [реестре Информация о методах сервиса](#). При этом концептуальное описание API (кто потребитель, ключевые возможности) должны быть описаны в технической документации.

Стандарт диаграмм

Для создания диаграммы используйте [Draw.io](#). Перед созданием схем или диаграмм рекомендуем ознакомиться с [краткой инструкцией по работе в Draw.io](#), скачать и установить актуальную [библиотеку элементов](#).

Все используемые в документе рисунки, схемы, диаграммы выкладываются в Материалы группы. Название рисунков, схем и диаграмм должно соответствовать шаблону: «Название статьи_рис N_название диаграммы».

Для того чтобы показать, на какие подсистемы разбивается разрабатываемая система, используется **диаграмма компонент**. Диаграмма рисуется на языке UML, используя [Draw.io](#) (!), а не StarUML и еще какой-то редактор.

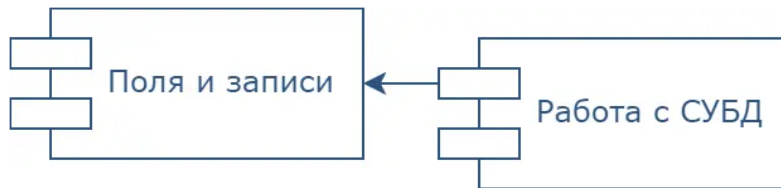


Рисунок 2. Диаграмма компонент

Для того чтобы наглядно показать алгоритм выполнения операции, используйте **диаграмму последовательностей** (sequence diagram).

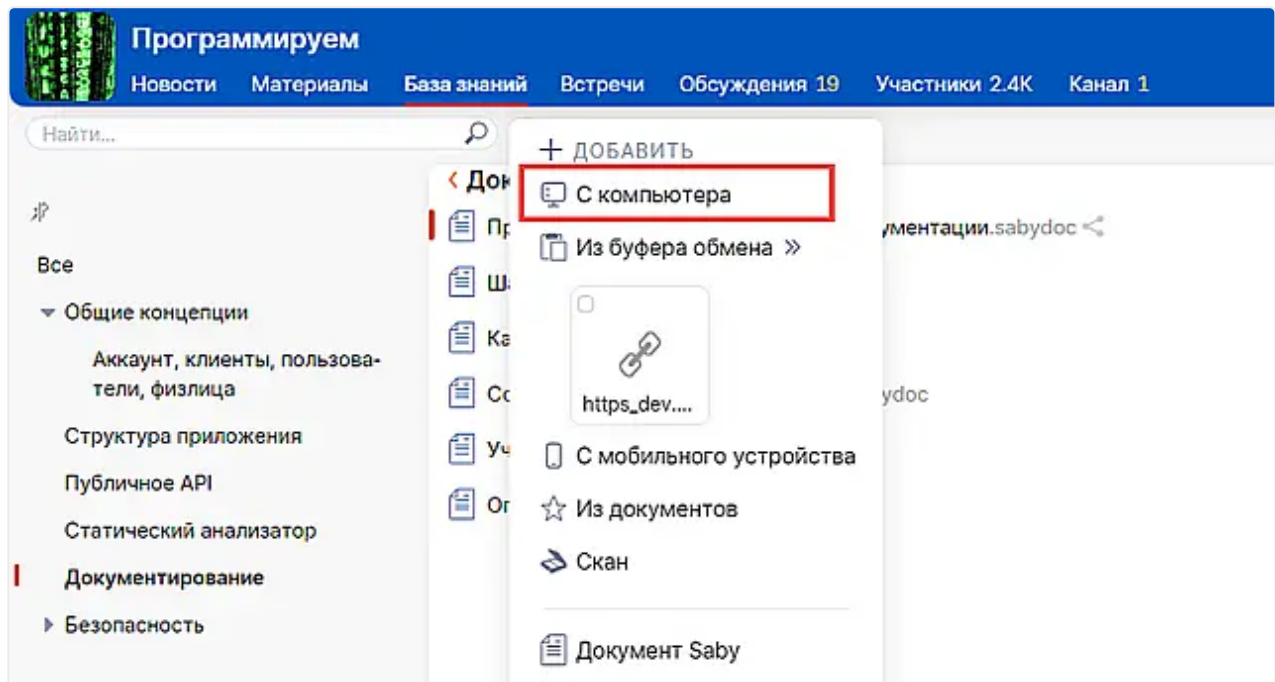
Объектами в диаграмме могут быть приложения, сервисы, модули и т.д. — главное, чтобы диаграмма отражала все задействованные участки и вызываемые методы API.

Пример: алгоритм просмотра кадрового документа в СБИС Архиве из online.sbis.ru.

"Переезд" документации с Диска компании в Базу знаний

Штатной возможности переместить документ с Диска компании в Базу знаний группы нет. Поэтому нужно:

1. Скачать документ с Диска компании на компьютер
2. В Базе знаний добавить статью "С компьютера"



При таком способе меняется ссылка, поэтому старый документ на СБИС Диске должен быть перенесен в Архив (**не удален!**) и в его шапке указано «Неактуальная редакция, см. <ссылка на актуальную статью в БЗ>».

Если переносите статью с wi.sbis.ru, то на wi также нужно организовать страницу со ссылками на актуальные статьи в Базе знаний.

Поиск...

Wasaby Framework

Настройка рабочего места

Разработка приложений

Архитектура

Инструменты разработки

Разработка сервиса

Разработка Saby приложения

Разработка интерфейса Wasaby

Слой совместимости WS3 и Wasaby

Разработка интерфейса WS3

Безопасность, Система прав

Кроссплатформенная разработка

Мобильная разработка

Разработка контроллера

Разработка iOS

Быстрый старт

Архитектура и основные принципы разработки в ко...

Инструменты

Про UI

Дополнительно

Разработка Android

Разработка стандартных компонентов

Пример разработки телефонного справочника в мобил...

Работа с кэшем в Firebase CloudFirestore

Как получить стек краша без сборки C++

Узел Agent мобильного приложения

Настройка параметров ini-файла из UI модулей прило...

Логирование

Интернационализация и локализация

Стандарты разработки

Middleware

Интеграция с внешними системами

Разработка мобильных приложений iOS

Внимание!

Материалы этого раздела находятся в Базе знаний группы [wasabyMobile](#).

Из этого раздела документации Вы узнаете о разработке мобильных приложений под iOS на Wasaby Framework.

Быстрый старт

Архитектура и основные принципы разработки в команде

Инструменты

Про UI

Дополнительно

Информация была полезной?

Заметили ошибку?

Остались вопросы? Задайте их на сервисе "Вопрос-ответ"

Задать вопрос

Вопрос E. A.

git-auto-merge